MINI PROYECTO DE LA UNIDAD 1

Clasificación de imágenes en CIFAR-10 con ResNet y EfficientNet

El mini proyecto consiste en un notebook de Google Colab que entrena y compara dos arquitecturas de clasificación de imágenes (ResNet-18 y EfficientNet-B0) sobre el dataset CIFAR-10. El notebook debe incluir el entrenamiento de ambos modelos, el cálculo de métricas de evaluación sobre el mismo conjunto de prueba y un reporte de diseño con las decisiones tomadas durante el desarrollo.

El alcance del proyecto es exclusivamente clasificación: cada imagen recibe una sola etiqueta de clase entre las 10 categorías de CIFAR-10. El proyecto no incluye detección de objetos ni segmentación; esa distinción es relevante para la sección 5 de este documento.

1. Librerías requeridas

El notebook debe importar las siguientes librerías. Ninguna es opcional.

Librería	Uso específico en este proyecto
torch	Tensores, autograd y funciones de pérdida (nn.CrossEntropyLoss)
torchvision	Acceso a CIFAR-10 (torchvision.datasets) y a las arquitecturas ResNet y EfficientNet (torchvision.models)
torchmetrics	Cálculo de accuracy, precisión, recall y F1 (sección 5)
matplotlib	Gráficas de curvas de entrenamiento y matrices de confusión
pandas	Tabla del historial de entrenamiento y tabla comparativa final
time	Medición de la duración de cada época de entrenamiento

2. Especificación técnica del modelo

El proyecto usa la implementación oficial de **torchvision.models** para ambas arquitecturas. No se permite una reimplementación manual de ResNet ni de EfficientNet: el objetivo del proyecto es la comparación experimental, no la reconstrucción de la arquitectura.

2.1 Modificación: última capa

El único cambio estructural permitido sobre cada arquitectura es el reemplazo de la última capa, para que la salida tenga 10 valores (uno por clase de CIFAR-10) en vez de 1000 (clases de ImageNet). El resto de la arquitectura, incluida la secuencia de bloques convolucionales revisada en el tema 1.3, permanece sin modificación.

```
modelo = torchvision.models.resnet18(weights="DEFAULT")
modelo.fc = torch.nn.Linear(modelo.fc.in_features, 10)

modelo = torchvision.models.efficientnet_b0(weights="DEFAULT")
modelo.classifier[1] = torch.nn.Linear(modelo.classifier[1].in_features, 10)
```

2.2 Transfer learning

Está permitido el uso de pesos preentrenados en ImageNet (parámetro **weights="DEFAULT"**). El transfer learning se formaliza como tema en la Unidad 3; en este proyecto su uso es opcional, sin embargo la elección debe declararse en el reporte de diseño.

2.3 Entrada del modelo

CIFAR-10 entrega imágenes de 32×32 píxeles. ResNet-18 y EfficientNet-B0 esperan una entrada de 224×224 píxeles. El notebook debe redimensionar cada imagen a 224×224 antes de introducirla al modelo, con **torchvision.transforms.Resize**. Si se usa transfer learning, la normalización de la imagen debe corresponder a las estadísticas de ImageNet (media 0.485, 0.456, 0.406; desviación estándar 0.229, 0.224, 0.225), no a las de CIFAR-10.

3. Condiciones del entrenamiento actividad en pares

La comparación entre ResNet-18 y EfficientNet-B0 solo es válida si ambos modelos se entrenan bajo las mismas condiciones. El notebook debe fijar, para los dos modelos, el mismo valor en cada uno de los siguientes puntos:

- El mismo subconjunto de datos de entrenamiento y de validación.
- El mismo número de épocas.
- El mismo optimizador y el mismo valor de learning rate.
- El mismo tamaño de lote (batch size).

4. Métricas de evaluación

El notebook debe reportar, para cada modelo y sobre el mismo conjunto de prueba, las siguientes cuatro métricas, calculadas con **torchmetrics** (promedio macro entre las 10 clases):

- Accuracy (exactitud)
- Precisión (precision)
- Recall (sensibilidad)
- F1-score

Además de las cuatro métricas anteriores, el notebook debe incluir la matriz de confusión de cada modelo sobre el conjunto de prueba, calculada con **torchmetrics.classification.MulticlassConfusionMatrix**.

4.1 Por qué IoU no se usa en este proyecto

IoU (Intersection over Union) mide la coincidencia espacial entre dos regiones: una caja delimitadora o una máscara predicha, contra una caja o máscara real. Esa métrica requiere que el modelo produzca una región, una localización dentro de la imagen.

CIFAR-10 es un dataset de clasificación: cada imagen tiene una sola etiqueta de clase, sin coordenadas de caja ni máscara de segmentación asociadas. ResNet-18 y EfficientNet-B0, en la configuración de este proyecto, producen una clase por imagen, no una región. Por esta razón, IoU no aplica a este proyecto y no debe incluirse en el notebook.

5. Entregables

El notebook final debe incluir, en este orden, cada uno de los siguientes elementos:

- Carga de CIFAR-10 y definición del subconjunto de entrenamiento, validación y prueba.
- Construcción de ResNet-18 y EfficientNet-B0 con torchvision.models, con la última capa adaptada a 10 clases.

- Entrenamiento de ambos modelos bajo las mismas condiciones (sección 4), con registro del historial en una tabla de pandas.
- Gráfica de las curvas de entrenamiento (loss y accuracy, train y validación) de ambos modelos.
- Tabla comparativa final con accuracy, precisión, recall y F1 de ambos modelos sobre el conjunto de prueba.
- Matriz de confusión de cada modelo sobre el conjunto de prueba.
- Reporte de diseño (sección 7 de este documento), completado dentro del propio notebook.

5.1 Reporte de diseño: contenido mínimo

El reporte de diseño, dentro del notebook, debe responder de forma explícita cada uno de los siguientes puntos:

N.º	Contenido requerido
1	Arquitectura recomendada por el equipo para este problema, con justificación basada en los resultados obtenidos, no solo en la teoría
2	Lista completa de hiperparámetros usados: learning rate, optimizador, número de épocas, batch size, tamaño del subconjunto de entrenamiento, uso o no de pesos preentrenados
3	Hardware de entrenamiento (CPU o GPU) y duración registrada de cada época
4	Diagnóstico de las curvas de entrenamiento de cada modelo: ajuste saludable, sobreajuste (overfitting) o subajuste (underfitting), con la evidencia correspondiente
5	Relación entre número de parámetros de cada arquitectura y su desempeño final (capacidad frente a eficiencia)
N.º	**Contenido requerido**
6	Par de clases con mayor confusión en cada matriz de confusión, con una explicación razonada

6. Criterios de evaluación

Criterio	Descripción	Puntos
Implementación técnica	El notebook usa correctamente torchvision.models, con la última capa adaptada según la sección 3, y se ejecuta sin errores	40
Rigor experimental	Ambos modelos se entrenan bajo las mismas condiciones (sección 4); los hiperparámetros están documentados	25
Métricas y curvas	Las cuatro métricas, las curvas y las matrices de confusión están completas para ambos modelos	20
Reporte de diseño	Los seis puntos de la sección 6.1 están respondidos con evidencia extraída del propio notebook	15

Total: 100 puntos.

---

7. Bibliografía y recursos de referencia

- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. arXiv:1512.03385.

- Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. ICML 2019.
- TorchMetrics documentation — Classification: Accuracy. torchmetrics.readthedocs.io/en/v0.9.1/classification/accuracy.html
- PyTorch documentation — torchvision.models. pytorch.org/vision/stable/models.html
- Katanforoosh, K., & Ng, A. Stanford CS230: Deep Learning — Section 8, "Advanced Evaluation Metrics". cs230.stanford.edu/section/8
- Ultralytics. Consejos y buenas prácticas para entrenar modelos. docs.ultralytics.com/es/guides/model-training-tips
- Ayyadevara, K., & Reddy, Y. (2024). Modern Computer Vision with PyTorch. Packt Publishing.